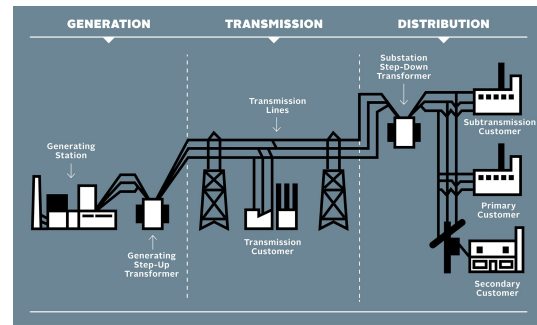


Quantization Evaluation Pipeline: Development, Experiments, and Lessons Learned

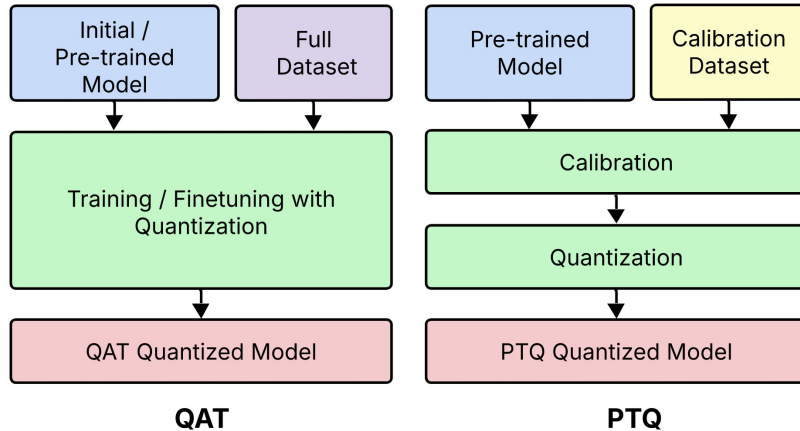
Gil Benezer, Brent Garey, Ryon Sajnovsky

Background

- Significant resources have been invested in making large generative AI models smaller
 - Pruning
 - Quantization
- Minimal work focuses on quantization for making small regression models smaller and faster



What is Quantization?



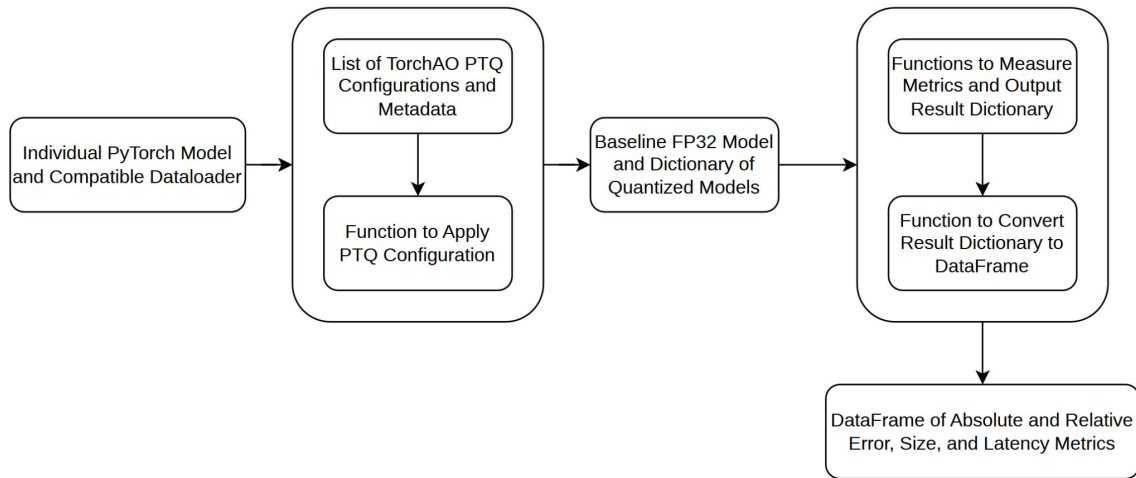
- Storage of weights and inference conducted in lower precision than standard 32 bit floating point
- Many different variables to modulate compression and performance
- Difficult to *a priori* know which approach would be best for a given application

Project Aims

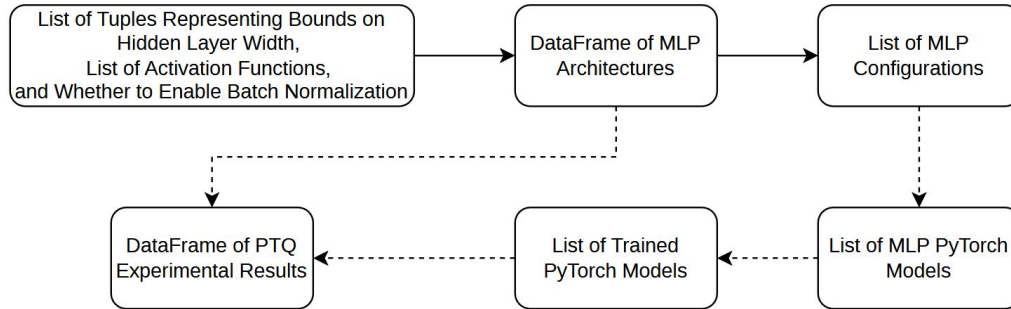
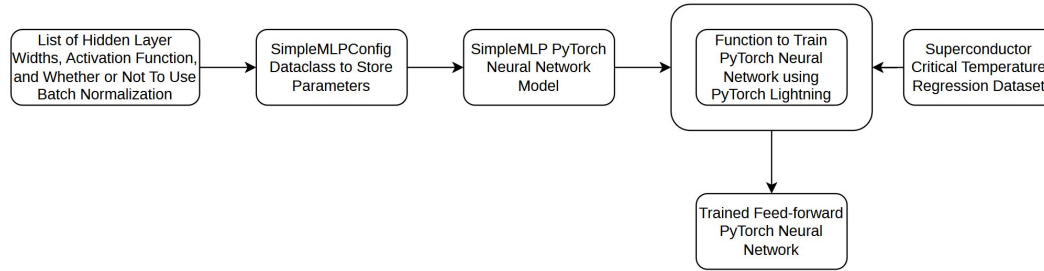
- Develop a tool to allow others to evaluate small regression models with respect to their performance after post-training quantization
 - Mean absolute error
 - Model size
 - Median latency
- Evaluate baseline performance of the tool using a feedforward neural networks
- Show the utility of the framework by evaluating trends in feedforward neural network width and activation function upon applying PTQ

Main Pipeline

- Given a PyTorch Model and compatible dataloader, output a DataFrame of metrics
 - Uses TorchAO library for PTQ using default arguments



Auxiliary Pipelines

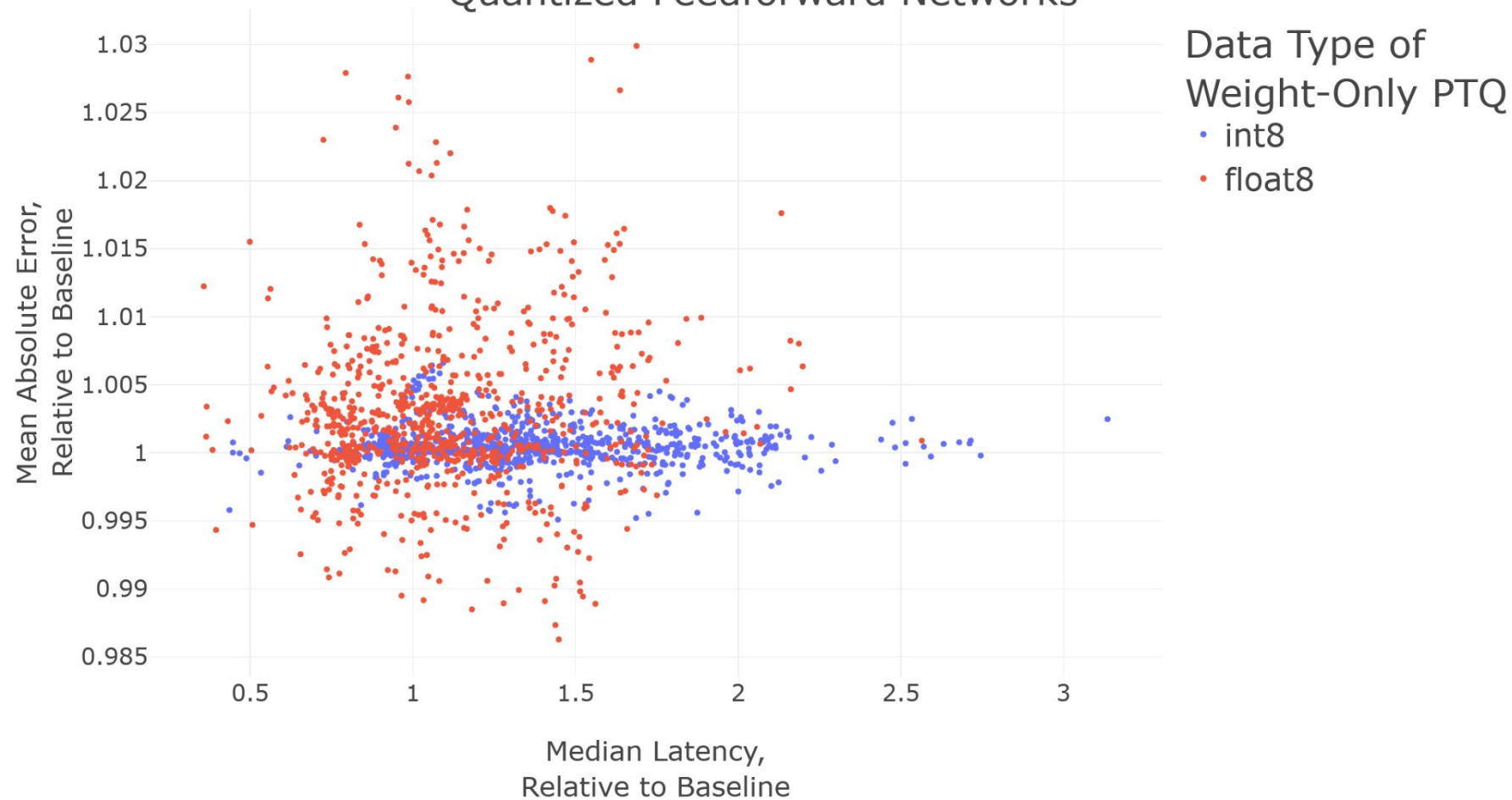


- Needed to build pipelines for constructing feedforward neural networks
- Networks were trained to predict the critical temperature at which superconductivity appears in various materials
- Visualization was executed with custom scripts instead of built into the pipeline

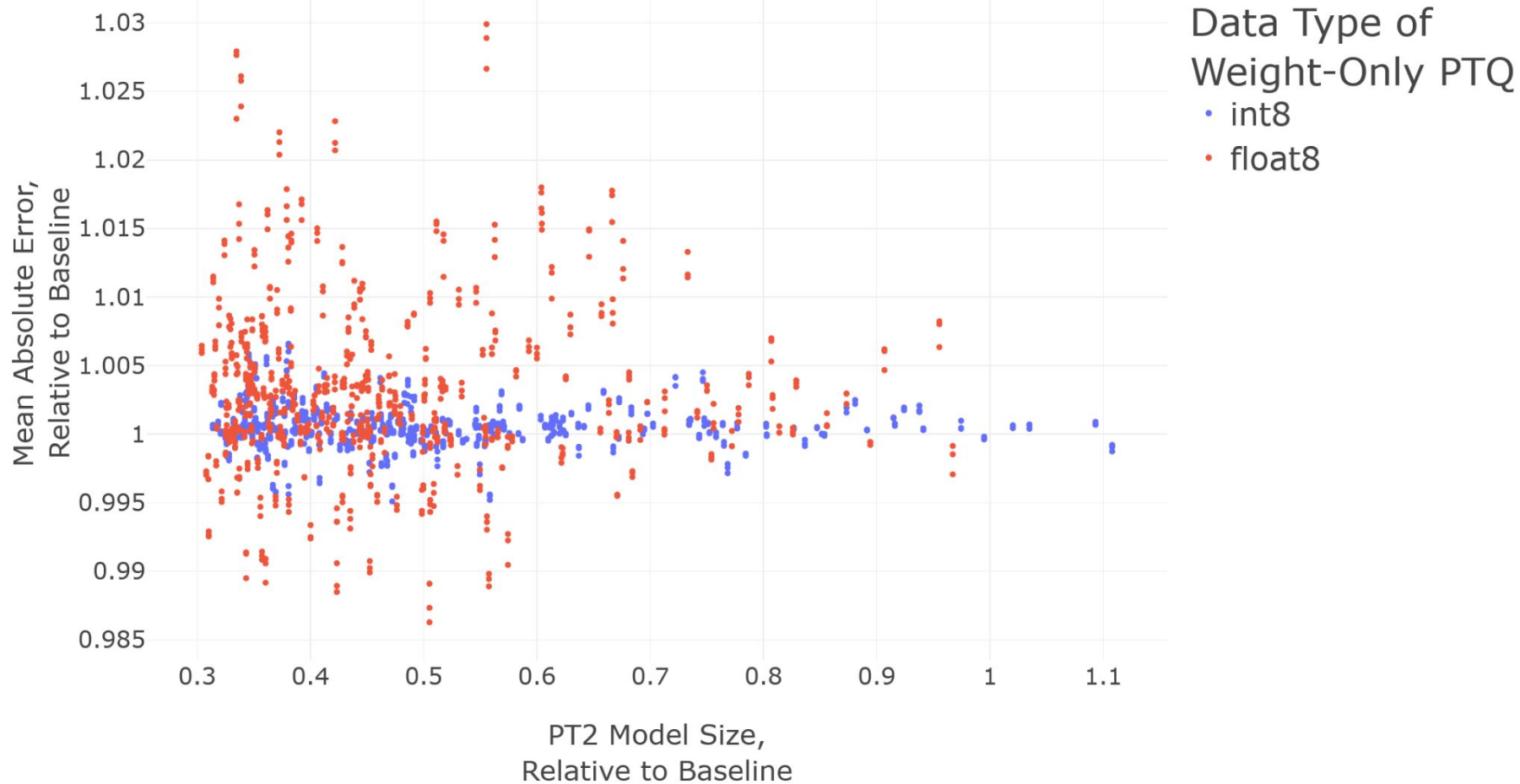
Architecture Evaluation

- Model Architecture:
 - Hidden Layer Width Bounds: [(128, 1024), (64, 512), (32, 256)]
 - Activation Functions: ReLU, Leaky ReLU, GELU, ELU, CELU
- 281 models were trained for 25 epochs and PTQ evaluation was run 3 times on each
- Execution environment: Northeastern Research Cluster NVIDIA V100
- Focus was on weight-only configuration results
 - Weight-only configurations and the Int8 dynamic calibration were the only TorchAO configurations that consistently worked

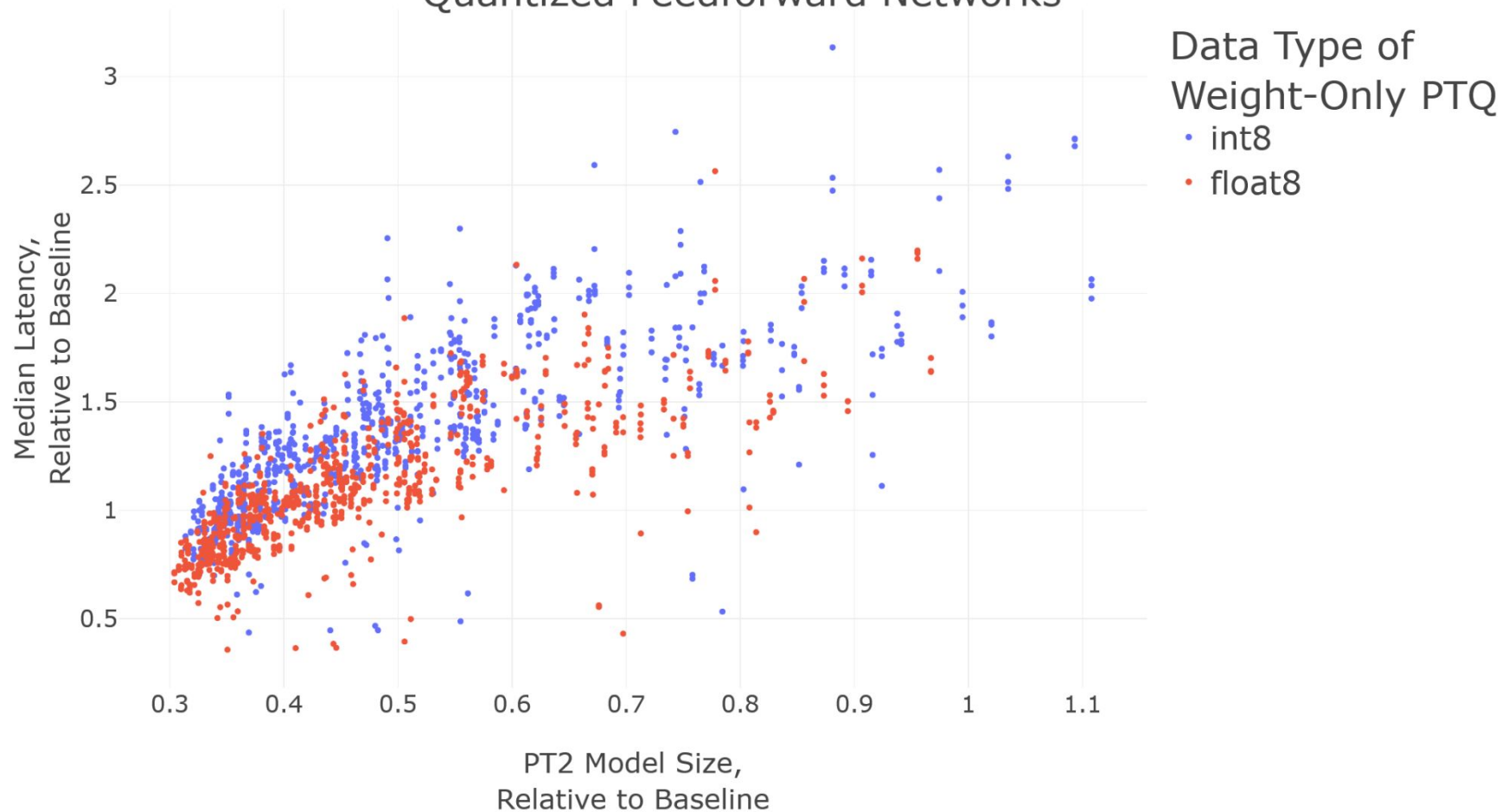
Latency-Error Pareto Frontier of Quantized Feedforward Networks



Compression-Error Pareto Frontier of Quantized Feedforward Networks



Compression-Latency Pareto Frontier of Quantized Feedforward Networks



Lessons Learned

- TorchAO is not an ideal solution for the basis of such a tool
 - Configurations fail in bizarre or unexpected ways
 - Not compatible with edge device PyTorch framework ExecuTorch
 - Poor support for QAT and static calibration configurations
- Within the architecture design space we examined
 - Activation function had a meaningful impact on model error in an absolute sense
 - There was not a meaningful tradeoff between model error and other performance metrics
 - Float8 quantized models had a higher variance in model error relative to Int8 quantization

Summary

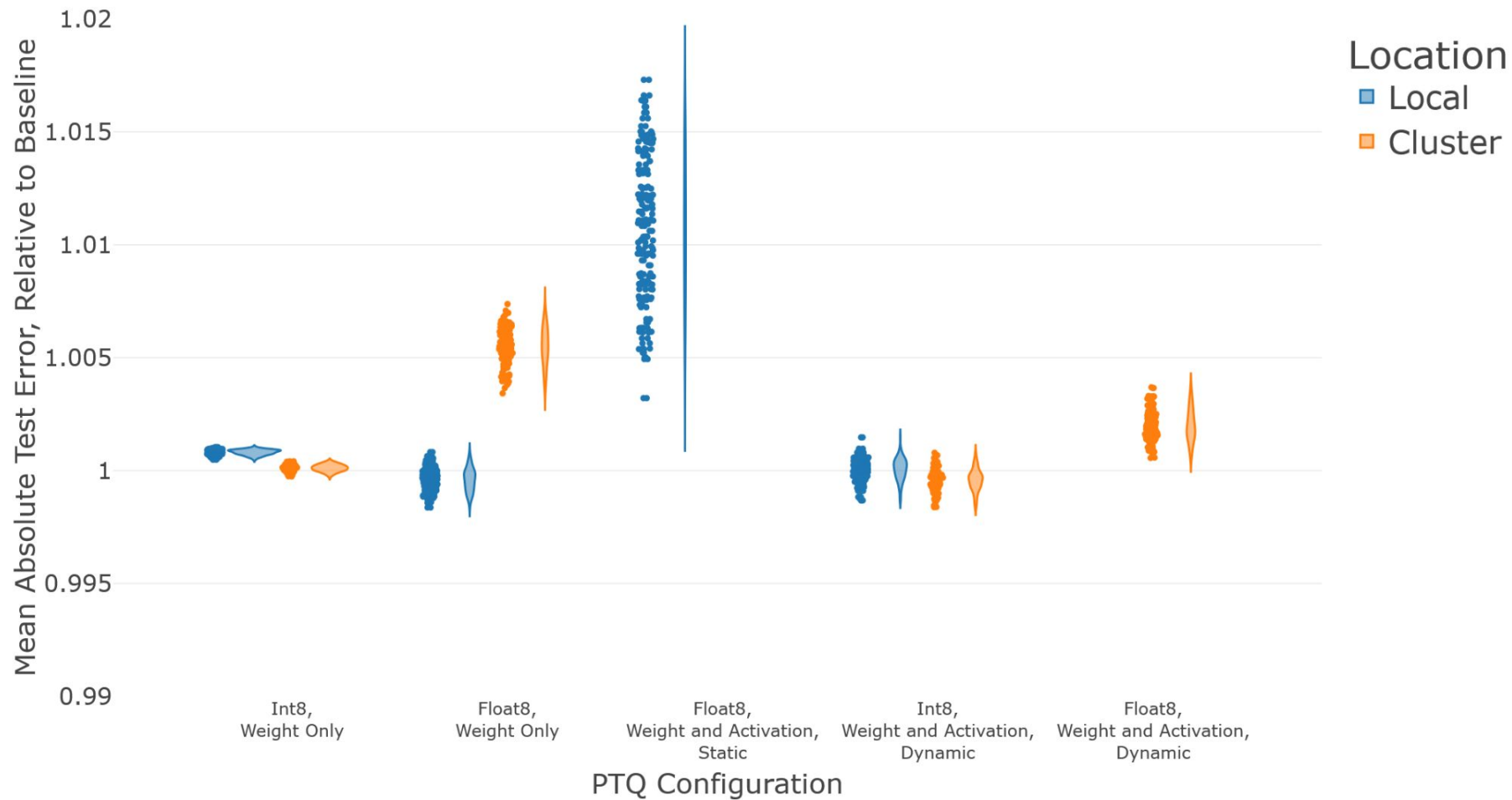
- We were able to successfully make a pipeline to quantize regression neural networks for PTQ configuration evaluation
- We were able to evaluate the base functionality of the pipeline in multiple execution environments
- We were able to conduct an experiment looking at feedforward network architecture performance with the quantization pipeline

Backup Slides

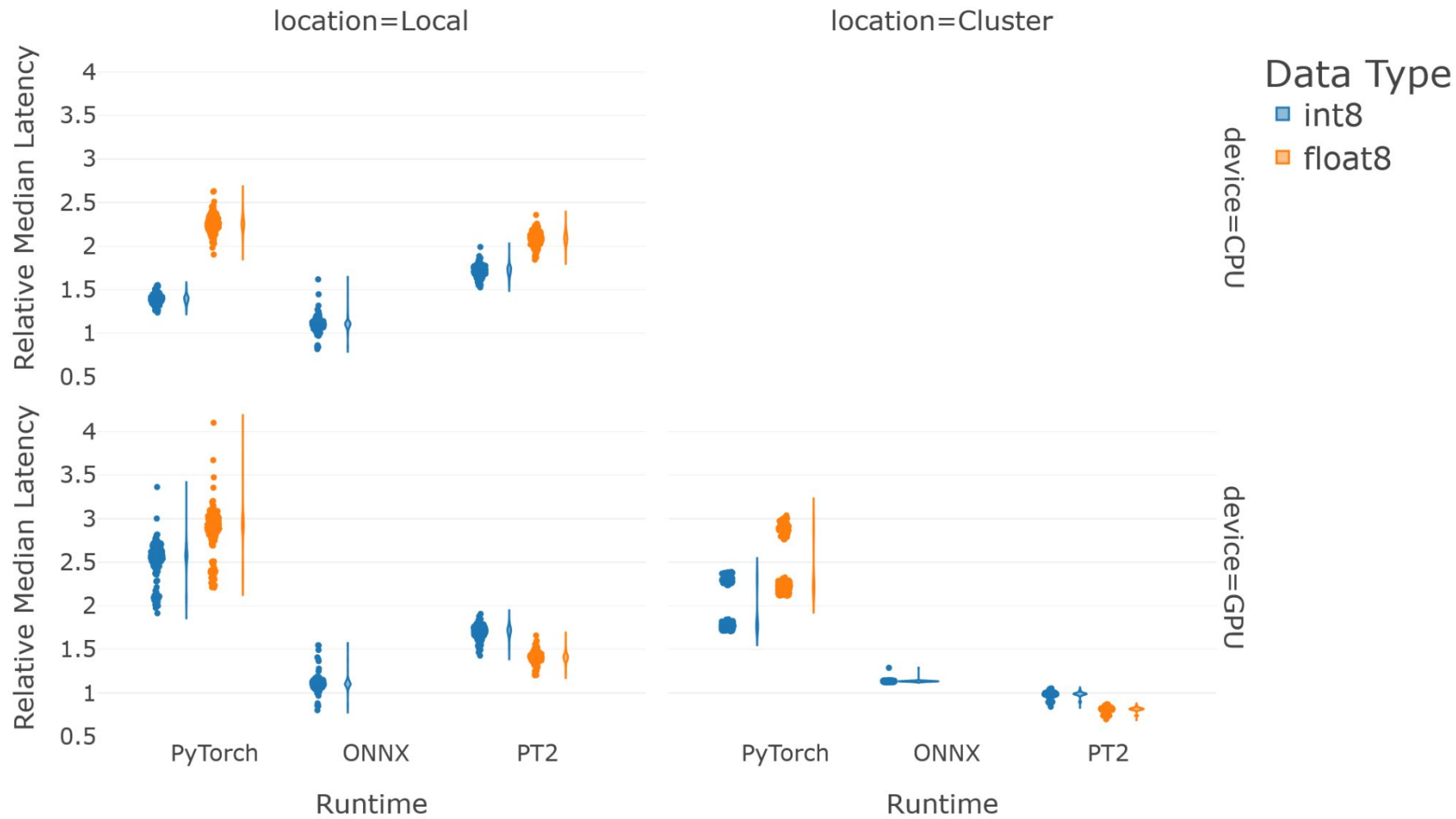
Baseline Evaluation

- Model Architecture:
 - Hidden Layer Widths: 512, 256, 128
 - Activation Function: Gaussian Error Linear Unit
- Model was trained 10 times for 25 epochs and PTQ evaluation was run 10 times per training run
 - 100 measurements of each metric
- Execution environments
 - Local CPU
 - Local NVIDIA GeForce RTX 3070 Laptop GPU
 - Northeastern Research Cluster NVIDIA H200
- Weight-only configurations were also evaluated in ONNX and PT2 file formats
- Differences between local GPU and CPU were negligible after running

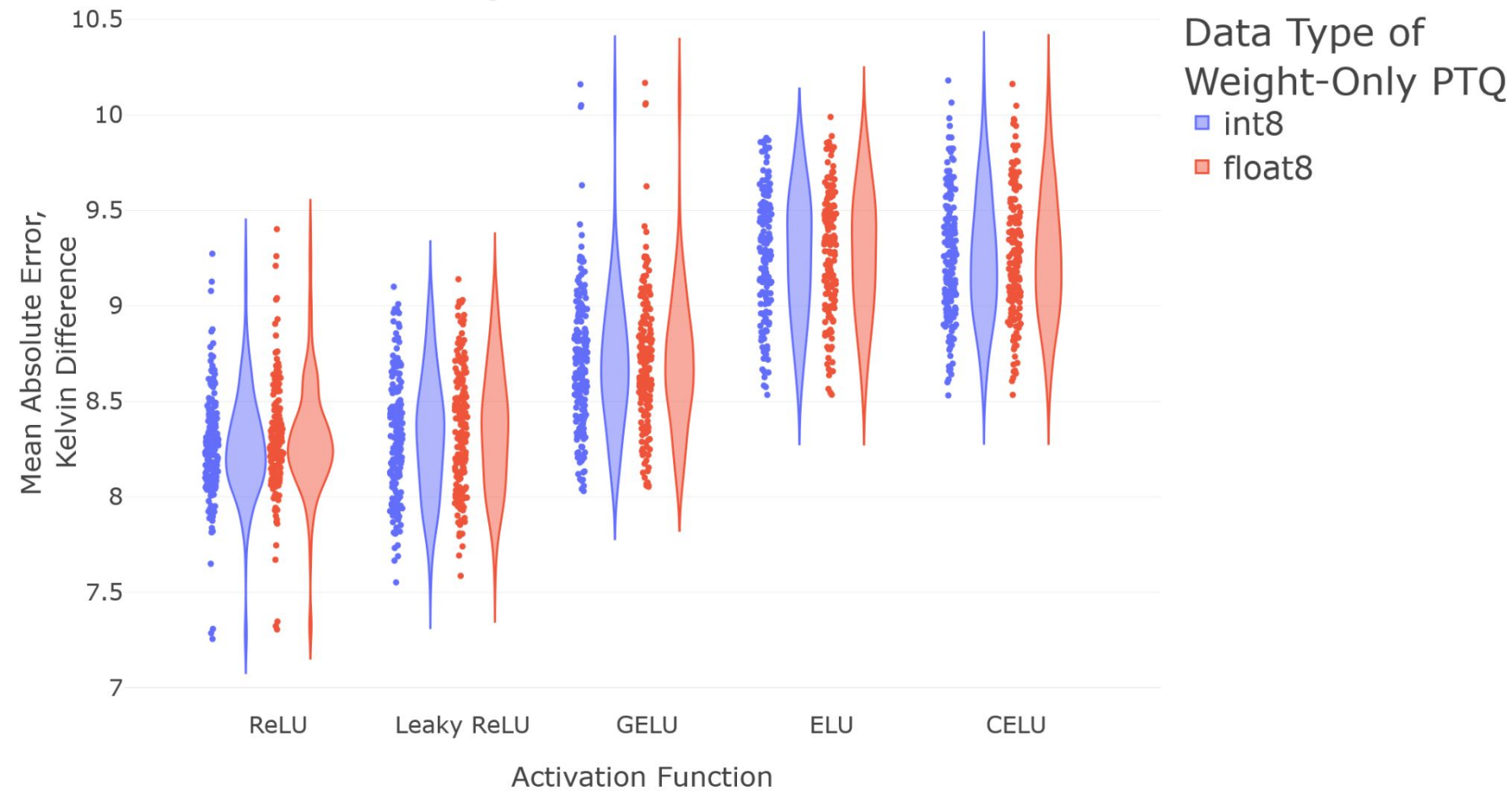
PTQ Approaches Differ With Respect to Test Error



Relative Median Inference Latency Variation Between Runtimes



Mean Absolute Error Versus Activation Functions in Quantized Feedforward Networks



Relative Error Versus Activation Functions in Quantized Feedforward Networks

